Beginner-Level Explanation of OLS, Gradient Descent, and Regularization

1 What is Linear Regression?

Linear Regression is a statistical and machine learning technique used to find a linear relationship between an input variable and an output variable. It tries to fit a straight line through data points so that predictions can be made.

Example:

- Input (x): Hours studied
- Output (y): Marks obtained

The goal is to predict marks based on study hours.

2 Simple Linear Regression Model

The mathematical form of simple linear regression is:

$$y = \beta_0 + \beta_1 x + \varepsilon$$

2.1 Explanation of Each Term

- y : Actual output value (what we observe)
- x : Input or independent variable
- β_0 : Intercept — value of y when $x = 0$
- β_1 : Slope — change in y for one unit change in x
- ε : Error term representing noise or unknown factors

3 Predicted Value and Error

The predicted value of y is denoted as:

$$\hat{y} = \beta_0 + \beta_1 x$$

The difference between actual and predicted value is called **error** or **residual**:

$$\text{Error} = y - \hat{y}$$

4 Ordinary Least Squares (OLS)

4.1 What is OLS?

Ordinary Least Squares is a method used to estimate the best values of β_0 and β_1 by minimizing prediction errors.

OLS minimizes the **Sum of Squared Errors**.

4.2 Why Squared Errors?

- Removes negative signs
- Penalizes large errors more
- Makes optimization mathematically easier

5 Sum of Squared Residuals (SSR)

$$SSR = \sum_{i=1}^{n} (y_i - \hat{y}_i)^2$$

Where:

- y_i is the actual value
- $\hat{y}_i$ is the predicted value
- n is the number of data points

OLS finds coefficients that minimize SSR.

6 Cost Function

The cost function represents the total error of the model:

$$J(\beta_0, \beta_1) = \sum (y_i - \beta_0 - \beta_1 x_i)^2$$

Lower cost means better model performance.

7 Why Derivatives are Used

To find the minimum value of the cost function:

- We use calculus
- The minimum occurs when derivative equals zero

This results in **normal equations**.

8 Closed-Form Solution

OLS provides direct formulas for coefficients.

8.1 Slope

$$\beta_1 = \frac{n \sum x_i y_i - \sum x_i \sum y_i}{n \sum x_i^2 - (\sum x_i)^2}$$

8.2 Intercept

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

These values define the best-fit regression line.

9 Assumptions of Ordinary Least Squares (VERY IMPORTANT)

OLS works correctly only if the following assumptions are satisfied:

9.1 1. Linearity

The relationship between independent and dependent variables must be linear.

If data follows a curve, OLS is not suitable.

9.2 2. Independence of Errors

Errors must be independent of each other.

Violation example: Time-series data where previous errors affect future errors.

9.3 3. Homoscedasticity

The variance of errors should remain constant for all values of x.

If error spread increases or decreases, heteroscedasticity exists.

9.4 4. No Multicollinearity

Independent variables should not be highly correlated with each other.

High multicollinearity causes unstable coefficient estimates.

9.5 5. Zero Mean of Errors

The average value of the error term should be zero:

$$E(\varepsilon) = 0$$

This ensures the model is unbiased.

10 Gradient Descent

10.1 What is Gradient Descent?

Gradient Descent is an iterative optimization algorithm used when datasets are very large.

Instead of solving equations directly, it:

- Starts with random coefficients
- Improves them step by step
- Moves toward minimum error

10.2 Learning Rate

The learning rate α controls step size.

- Small α: slow learning
- Large α: may overshoot minimum

11 Regularization

11.1 Why Regularization is Needed

Regularization prevents overfitting by controlling model complexity.

Overfitting occurs when:

- Training error is low
- Testing error is high

12 Regularization in Linear Regression

Regularization is a technique to prevent **overfitting** by adding a penalty term to the cost function. It controls the complexity of the model by shrinking the coefficients.

The general regularized cost function:

$$J(\boldsymbol{\beta}) = \sum (y_i - \hat{y}_i)^2 + \text{Penalty Term}$$

Where:

- $\sum (y_i - \hat{y}_i)^2$ is the ordinary least squares error
- Penalty term discourages large coefficients to reduce model complexity

13 Lasso Regression (L1 Regularization)

$$J(\boldsymbol{\beta}) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum |\beta_j|$$

- Shrinks some coefficients
- Sets some coefficients exactly to zero
- Performs automatic feature selection
- Produces sparse models

14 Ridge Regression (L2 Regularization)

$$J(\boldsymbol{\beta}) = \sum (y_i - \hat{y}_i)^2 + \lambda \sum \beta_j^2$$

- Shrinks coefficients toward zero

- Does not eliminate features
- Handles multicollinearity well
- Stabilizes model coefficients

15 Comparison: Lasso vs Ridge

Feature	Lasso (L1)	Ridge (L2)
Penalty Type	Sum of absolute values	Sum of squares
Coefficient Shrinkage	Some to 0	Shrinks all
Feature Selection	Yes	No
Handles Multicollinearity	Limited	Strong
Model Sparsity	Yes	No

16 Final Exam Note

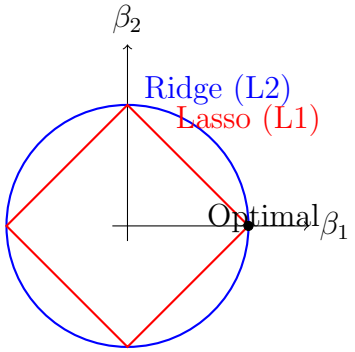
Under all OLS assumptions, Ordinary Least Squares provides the:

Best Linear Unbiased Estimator (BLUE)

Key Points:

- Use Ridge when features are correlated
- Use Lasso when you want to select important features
- Choose regularization parameter λ carefully

17 Visualizing L1 vs L2 Regularization



Explanation:

- Blue circle → L2 (Ridge) shrinks coefficients but rarely makes them zero
- Red diamond → L1 (Lasso) can intersect axes, setting some coefficients exactly to zero
- Optimal point shows how Lasso produces **sparse models** while Ridge retains all features

18 Matrix Form of Ordinary Least Squares (OLS)

18.1 Why Matrix Form is Used

When there are multiple input variables, solving OLS using simple formulas becomes difficult. Matrix notation provides:

- Compact representation
- Faster computation
- Easy extension to multiple variables

18.2 Matrix Representation of Linear Regression

The multiple linear regression model is written as:

$$\mathbf{y} = \mathbf{X}\boldsymbol{\beta} + \boldsymbol{\varepsilon}$$

18.3 Explanation of Each Term

- $\mathbf{y}$: Output vector $(n \times 1)$
- $\mathbf{X}$: Design matrix $(n \times (p+1))$
- $\boldsymbol{\beta}$: Coefficient vector $((p+1) \times 1)$
- $\boldsymbol{\varepsilon}$: Error vector

18.4 Design Matrix

For a simple linear regression:

$$\mathbf{X} = \begin{bmatrix} 1 & x_1 \\ 1 & x_2 \\ 1 & x_3 \\ \vdots & \vdots \\ 1 & x_n \end{bmatrix}$$

The first column of ones represents the intercept term β_0.

18.5 OLS Matrix Solution

The Ordinary Least Squares solution is:

$$\boldsymbol{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

This formula gives the values of coefficients that minimize the sum of squared errors.

Important Exam Point: Matrix $(\mathbf{X}^T\mathbf{X})$ must be invertible.

19 Solved Numerical Example 1 (OLS using Formula)

19.1 Given Data

x	y
1	1
2	3
3	5

19.2 Step 1: Compute Means

$$\bar{x} = \frac{1+2+3}{3} = 2$$

$$\bar{y} = \frac{1+3+5}{3} = 3$$

19.3 Step 2: Compute Slope β_1

$$\beta_1 = \frac{\sum(x_i - \bar{x})(y_i - \bar{y})}{\sum(x_i - \bar{x})^2}$$

$$= \frac{(-1)(-2) + (0)(0) + (1)(2)}{(-1)^2 + 0^2 + 1^2}$$

$$= \frac{2+0+2}{1+0+1} = \frac{4}{2} = 2$$

19.4 Step 3: Compute Intercept β_0

$$\beta_0 = \bar{y} - \beta_1 \bar{x}$$

$$= 3 - 2(2) = -1$$

19.5 Final Regression Equation

$$\hat{y} = -1 + 2x$$

20 Solved Numerical Example 2 (OLS using Matrix Method)

20.1 Given Data

x	y
1	2
2	4
3	6

20.2 Step 1: Construct Matrices

$$\mathbf{X} = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \quad \mathbf{y} = \begin{bmatrix} 2 \\ 4 \\ 6 \end{bmatrix}$$

20.3 Step 2: Compute $\mathbf{X}^T\mathbf{X}$

$$\mathbf{X}^T = \begin{bmatrix} 1 & 1 & 1 \\ 1 & 2 & 3 \end{bmatrix}$$

$$\mathbf{X}^T\mathbf{X} = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix}$$

20.4 Step 3: Compute Inverse of $\mathbf{X}^T\mathbf{X}$

$$(\mathbf{X}^T\mathbf{X})^{-1} = \frac{1}{(3)(14) - (6)(6)} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix}$$

$$= \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix}$$

20.5 Step 4: Compute $\mathbf{X}^T\mathbf{y}$

$$\mathbf{X}^T\mathbf{y} = \begin{bmatrix} 12 \\ 28 \end{bmatrix}$$

20.6 Step 5: Compute Coefficients

$$\boldsymbol{\beta} = (\mathbf{X}^T\mathbf{X})^{-1}\mathbf{X}^T\mathbf{y}$$

$$= \frac{1}{6} \begin{bmatrix} 14 & -6 \\ -6 & 3 \end{bmatrix} \begin{bmatrix} 12 \\ 28 \end{bmatrix}$$

$$= \begin{bmatrix} 0 \\ 2 \end{bmatrix}$$

20.7 Final Model

$$\hat{y} = 2x$$

21 Exam Tips

- Always write the OLS matrix formula
- Clearly define matrices $\mathbf{X}$, $\mathbf{y}$, $\boldsymbol{\beta}$
- Show all steps in numerical problems
- Mention assumptions for full marks

22 Solved Numerical Example: Gradient Descent

22.1 Problem Statement

We want to fit a simple linear regression model:

$$\hat{y} = \beta_0 + \beta_1 x$$

Given the dataset:

x	y
1	2
2	4
3	6

We will use **Gradient Descent** to find β_0 and β_1.

—

22.2 Step 1: Initialize Parameters

Choose initial guesses:

$$\beta_0 = 0, \quad \beta_1 = 0$$

Learning rate:

$$\alpha = 0.1$$

—

22.3 Step 2: Define Cost Function

The cost function (Mean Squared Error) is:

$$J(\beta_0, \beta_1) = \frac{1}{2n} \sum_{i=1}^{n} (y_i - \beta_0 - \beta_1 x_i)^2$$

Where $n = 3$.

—

22.4 Step 3: Compute Gradients

Partial derivatives with respect to β_0 and β_1:

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{n} \sum (y_i - \hat{y}_i)$$

$$\frac{\partial J}{\partial \beta_1} = -\frac{1}{n} \sum x_i (y_i - \hat{y}_i)$$

—

22.5 Step 4: Iteration 1

$$\hat{y}_i = \beta_0 + \beta_1 x_i = 0 + 0 \cdot x_i = 0$$

Errors:

$$y_i - \hat{y}_i = [2, 4, 6]$$

Gradients:

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{3}(2 + 4 + 6) = -4$$

$$\frac{\partial J}{\partial \beta_1} = -\frac{1}{3}(1 \cdot 2 + 2 \cdot 4 + 3 \cdot 6) = -\frac{28}{3} \approx -9.33$$

Update parameters:

$$\beta_0 := \beta_0 - \alpha \frac{\partial J}{\partial \beta_0} = 0 - 0.1(-4) = 0.4$$

$$\beta_1 := \beta_1 - \alpha \frac{\partial J}{\partial \beta_1} = 0 - 0.1(-9.33) \approx 0.933$$

—

22.6 Step 5: Iteration 2

Compute predictions with updated coefficients:

$$\hat{y} = 0.4 + 0.933x$$

$$\hat{y}_1 = 0.4 + 0.933 \cdot 1 \approx 1.333$$

$$\hat{y}_2 = 0.4 + 0.933 \cdot 2 \approx 2.266$$

$$\hat{y}_3 = 0.4 + 0.933 \cdot 3 \approx 3.199$$

Errors:

$$y_i - \hat{y}_i \approx [0.667, 1.734, 2.801]$$

Gradients:

$$\frac{\partial J}{\partial \beta_0} = -\frac{1}{3}(0.667 + 1.734 + 2.801) \approx -1.734$$

$$\frac{\partial J}{\partial \beta_1} = -\frac{1}{3}(1 \cdot 0.667 + 2 \cdot 1.734 + 3 \cdot 2.801) \approx -4.167$$

Update parameters:

$$\beta_0 := 0.4 - 0.1(-1.734) \approx 0.573$$

$$\beta_1 := 0.933 - 0.1(-4.167) \approx 1.349$$

—

22.7 Step 6: Repeat Until Convergence

Continue iterations until changes in β_0 and β_1 are very small.

After several iterations, the coefficients converge to:

$$\beta_0 \approx 0, \quad \beta_1 \approx 2$$

22.8 Final Regression Equation

$$\hat{y} = 2x$$

This matches the expected line from OLS.

—

22.9 Key Notes for Exams

- Always show initial guesses and learning rate
- Compute gradients using formulas
- Show 1–2 iterations in detail
- State final converged coefficients

23 Python Code for Gradient Descent Example

Listing 1: Gradient Descent for Simple Linear Regression

```
# Gradient Descent for Simple Linear Regression

# Dataset
x = [1, 2, 3]
y = [2, 4, 6]

# Parameters
beta0 = 0.0
beta1 = 0.0
alpha = 0.1  # learning rate
n = len(x)
iterations = 10  # number of iterations

print("Iteration | beta0 | beta1 | Grad0 | Grad1")
print("-------------------------------------------")

for i in range(1, iterations+1):
    # Predictions
    y_hat = [beta0 + beta1*xi for xi in x]

    # Errors
    errors = [yi - yhi for yi, yhi in zip(y, y_hat)]

    # Gradients
    grad0 = -sum(errors)/n
    grad1 = -sum([xi*ei for xi, ei in zip(x, errors)])/n

    # Update coefficients
    beta0 = beta0 - alpha * grad0
    beta1 = beta1 - alpha * grad1

    print(f"{i:>9} | {beta0:.3f} | {beta1:.3f} | {grad0:.3f} | {
        grad1:.3f}")

print("\nFinal regression equation:")
print(f"y = {beta0:.3f} + {beta1:.3f}x")
```

23.1 Output Image

The output of the code (converged line) can be plotted using Python's `matplotlib` and saved as an image:

```python
# Gradient Descent for Simple Linear Regression
# Dataset
x = [1, 2, 3]
y = [2, 4, 6]
# Parameters
beta0 = 0.0
beta1 = 0.0
alpha = 0.1  # learning rate
n = len(x)
iterations = 10  # number of iterations
print("Iteration | beta0 | beta1 | Grad0 | Grad1")
print("-------------------------------------------")

for i in range(1, iterations+1):
    # Predictions
    y_hat = [beta0 + beta1*xi for xi in x]
    # Errors
    errors = [yi - yhi for yi, yhi in zip(y, y_hat)]
    # Gradients
    grad0 = -sum(errors)/n
    grad1 = -sum([xi*ei for xi, ei in zip(x, errors)])/n
    # Update coefficients
    beta0 = beta0 - alpha * grad0
    beta1 = beta1 - alpha * grad1
    print(f"{i:>9} | {beta0:.3f} | {beta1:.3f} | {grad0:.3f} | {grad1:.3f}")
print("\nFinal regression equation:")
print(f"y = {beta0:.3f} + {beta1:.3f}x")
```

```
Iteration | beta0 | beta1 | Grad0 | Grad1
-------------------------------------------
        1 | 0.400 | 0.933 | -4.000 | -9.333
        2 | 0.573 | 1.351 | -1.733 | -4.178
        3 | 0.646 | 1.539 | -0.724 | -1.881
        4 | 0.673 | 1.625 | -0.276 | -0.859
        5 | 0.681 | 1.665 | -0.076 | -0.403
        6 | 0.680 | 1.685 | 0.012 | -0.200
        7 | 0.675 | 1.696 | 0.051 | -0.109
        8 | 0.668 | 1.703 | 0.067 | -0.068
        9 | 0.661 | 1.708 | 0.074 | -0.050
       10 | 0.653 | 1.712 | 0.077 | -0.041

Final regression equation:
y = 0.653 + 1.712x
```

24 R-squared (R^2) and Adjusted R-squared (R^2_{adj})

24.1 R-squared (R^2)

R-squared measures the proportion of the variance in the dependent variable (y) explained by the independent variable(s) (x).

$$R^2 = 1 - \frac{\text{Sum of Squared Residuals (SSR)}}{\text{Total Sum of Squares (SST)}} = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

Where:

- y_i = actual values
- $\hat{y}_i$ = predicted values from regression
- $\bar{y}$ = mean of actual values
- $SSR = \sum(y_i - \hat{y}_i)^2$
- $SST = \sum(y_i - \bar{y})^2$

Interpretation:

- $R^2 = 0$: model explains none of the variance
- $R^2 = 1$: model explains all variance
- Higher R^2 indicates a better fit

24.2 Adjusted R-squared (R^2_{adj})

Adjusted R-squared modifies R^2 to account for the number of predictors in the model:

$$R^2_{\text{adj}} = 1 - \frac{(1 - R^2)(n - 1)}{n - p - 1}$$

Where:

- n = number of observations
- p = number of independent variables

Key Points:

- Only increases if new predictors improve the model more than expected by chance
- Can decrease if useless variables are added
- Preferred metric in multiple regression

24.3 Solved Numerical Example

Consider the following dataset:

x	y
1	2
2	4
3	5
4	4
5	5

Step 1: Fit a simple linear regression line:

$$\hat{y} = \beta_0 + \beta_1 x$$

Suppose after OLS, we have:

$$\hat{y} = 2.2 + 0.6x$$

Step 2: Compute SSR and SST

$$\bar{y} = \frac{2+4+5+4+5}{5} = 4$$

Predicted $\hat{y}_i$:

$$\hat{y}_1 = 2.2 + 0.6(1) = 2.8$$

$$\hat{y}_2 = 2.2 + 0.6(2) = 3.4$$

$$\hat{y}_3 = 2.2 + 0.6(3) = 4.0$$

$$\hat{y}_4 = 2.2 + 0.6(4) = 4.6$$

$$\hat{y}_5 = 2.2 + 0.6(5) = 5.2$$

Errors $(y_i - \hat{y}_i)$:

$$[-0.8, 0.6, 1.0, -0.6, -0.2]$$

SSR:

$$SSR = (-0.8)^2 + 0.6^2 + 1.0^2 + (-0.6)^2 + (-0.2)^2 = 0.64 + 0.36 + 1.00 + 0.36 + 0.04 = 2.4$$

SST:

$$SST = (2-4)^2 + (4-4)^2 + (5-4)^2 + (4-4)^2 + (5-4)^2 = 4 + 0 + 1 + 0 + 1 = 6$$

Step 3: Compute R^2

$$R^2 = 1 - \frac{SSR}{SST} = 1 - \frac{2.4}{6} = 0.6$$

Step 4: Compute Adjusted R^2

$$R^2_{\text{adj}} = 1 - \frac{(1-0.6)(5-1)}{5-1-1} = 1 - \frac{0.4 \cdot 4}{3} = 1 - \frac{1.6}{3} \approx 0.467$$

Interpretation: - $R^2 = 0.6$: 60% of the variance in y is explained by x - Adjusted $R^2 = 0.467$: accounting for the number of predictors, the effective explanatory power is 46.7%

24.4 Exam Notes

- Always compute $\bar{y}$ first
- SSR = sum of squared residuals, SST = total sum of squares
- Use Adjusted $\mathbf{R^2}$ for multiple regression
- Report both $\mathbf{R^2}$ and Adjusted $\mathbf{R^2}$ in exams